Claude Code LXC: Use and Best Practices

Oct 27, 2026 · Joe Solch

What you get

One Ubuntu 26.04 container per person, with Claude Code running as one normal user and never as root. Deploy a second container for a second person; containers are cheap, and people sharing one would also share sudo, /project and the web UI. Everything runs natively: there's no Docker in the container.

Part	Where it lives	Owned by
Claude Code	~/.local/bin/claude	the user
Claude settings, plugins, skills	~/.claude/	the user
Projects	/project	the user
Node.js, Python, Go, language servers	/usr, /usr/local	root
Rust toolchain	/opt/rust (your crates in ~/.cargo)	root
GitHub CLI (`gh`)	/usr/bin/gh, from GitHub's apt repository	root; you sign in as the user
Playwright browsers	/opt/ms-playwright	the user
Web UI (CloudCLI)	port 3001, service `cloudcli`	runs as the user
Update and health tools	`agentic-update` , `agentic-doctor` , `agentic-setup-user` , `agentic-errorlog`	root (run with sudo)
Problem log	~/Agentic-Errors.txt, only when something fails	the user
Phone alerts (optional)	ntfy, set in /etc/agentic/ntfy.conf	root

First steps after deploy

Do these right after the script finishes, in this order; step 1 matters most.

1. **Claim the web UI.** It listens only on the container unless you chose LAN at deploy. From your PC, run `ssh -L 3001:localhost:3001 <user>@<ip>` and open `http://localhost:3001` (with LAN access, open `http://<ip>:3001` instead). Create its login straight away: until you do, anyone who can reach it can create it.
2. **Log in as your user.** Use `ssh <user>@<ip>` with the user's own password, which the script requires to be different from root's. From the Proxmox host you can also run `pct enter <id>`, then `su -`

`<user>`. Root can log in over SSH with its password too, but don't run Claude as root.

3. **Sign in to Claude.** Run `claude` and follow `/login`. A claude.ai subscription also enables Remote Control.
4. **Check health.** Run `sudo agentic-doctor`. Every line should say OK. The only expected WARN is "logged in", and only before step 3. Any WARN or FAIL is also copied to ~/Agentic-Errors.txt, so if you ran it before step 3, delete that file.
5. **Set your git identity.** Run `git config --global user.name "..."` and `git config --global user.email "..."`.
6. **Sign in to GitHub.** Run `gh auth login` once (choose HTTPS and let it set up git credentials). Claude uses `gh` for pull requests and issues.
7. **Keep the install log.** It's on the Proxmox host under `/var/log/agentic-deploy/`, and inside the container at `/var/log/agentic-provision.log`.

Daily use

Log in as your user, `cd` into a project under /project, and run `claude` there. Claude reads that project's files and the shared /project/CLAUDE.md.

- **Start a project:** `mkdir -p /project/src/myapp && cd $_ && git init`, then `claude`.
- **Resume:** `claude --continue` picks up the last session in that folder; `claude --resume` lets you choose one.
- **Web UI:** chat, a file editor, git and a shell, all running as your user. By default it's reachable only through an SSH tunnel: `ssh -L 3001:localhost:3001 <user>@<ip>`, then `http://localhost:3001`. To open it to your LAN later, change `HOST=127.0.0.1` to `HOST=0.0.0.0` in /etc/systemd/system/cloudcli.service, then run `sudo systemctl daemon-reload && sudo systemctl restart cloudcli`.
- **Claude Desktop:** connect over SSH from your own PC; see the next section.
- **From your phone:** in a session, type `/rc` (Remote Control), then continue from the Claude app or claude.ai/code.
- **No Docker:** there are no containers inside the container. If a project needs a database server, run `sudo apt install postgresql` (or `redis-server`) yourself and use it as a normal service.
- **Admin commands:** run `sudo apt install ...` and other root tasks yourself. Claude is blocked from sudo and su, so when it needs one it should ask you.
- **Python:** use a venv per project (`python3 -m venv .venv`) rather than system-wide installs.
- **Node:** `npm i -g` installs into your own ~/.local, so no sudo is needed.

Working from your PC with Claude Desktop

You don't need a desktop VM. Claude Desktop on your own computer (Mac, Windows, or the Linux beta for Ubuntu and Debian) can open an SSH session to this container. It then works on the container's /project with your ~/.claude settings and skills.

1. **Make a key on your PC** if you don't have one: `ssh-keygen -t ed25519`.
2. **Copy it to the container:** `ssh-copy-id <user>@<ip>`. You can also give the key's path when you run the deploy script, which installs it for root and your user.
3. **Load it in your SSH agent** (`ssh-add`). Desktop can't type passwords or passphrases.
4. **Add the connection:** in Desktop's Code tab, add an SSH connection to `<user>@<ip>`, leaving the key field empty to use your SSH config and agent.
5. **Pick the folder** `/project/src/<app>` and start a session.

Good to know:

- On first connect, Desktop installs its own copy of Claude Code under `~/.claude/remote/`, so allow about 1 GB in your home folder.
- SFTP must be on; the script makes sure it is.
- The Browser pane doesn't work in an SSH session; forward the port instead (`ssh -L 3000:localhost:3000 <user>@<ip>`) and open it on your PC.
- `gh` is already installed on the container; sign in once with `gh auth login` (first steps, step 6) so Desktop can create pull requests.
- For editing by hand alongside Claude, VS Code's Remote-SSH extension works with the same key.

Source: Claude Code Desktop docs.

Maintenance

Updates run on their own every Sunday at 4 AM ET; you only step in when the health check fails.

When an update or health check hits a WARN or FAIL, an entry is appended to `Agentic-Errors.txt` in your home folder (the container has no desktop). Each entry has the time, the step that failed and the last 40 lines of the log. If the file is there, something needs a look; if it isn't, every run since you last deleted it was clean. Entries older than 90 days are trimmed and the file is capped at 256 KB. Delete it once you've dealt with the problems.

Task	Command	Notes
Update everything now	`sudo agentic-update`	OS, npm and the global dev tools (including Playwright and gopls), Rust, Claude Code and the web UI, then a health check
Health check	`sudo agentic-doctor`	Exit 0 means healthy; also warns if sudo stops asking for a password

Task	Command	Notes
Check for problems	`cat ~/Agentic-Errors.txt`	Only exists after a WARN or FAIL; delete it when dealt with
Test phone alerts	`sudo agentic-errorlog --test-ntfy`	Only if ntfy is set up; see the next section
Read the update log	`less /var/log/agentic-update.log`	Rotated monthly
Repair the Claude setup	`sudo agentic-setup-user`	Reinstalls Claude and plugins; keeps your settings.json
Web UI status and logs	`systemctl status cloudcli`, `journalctl -u cloudcli`	
Install logs	host: `/var/log/agentic-deploy/`; container: `/var/log/agentic-provision.log`	The container log includes a trace of every command

Before big changes, take a snapshot in Proxmox: `pct snapshot <id> before-change`. Back up /project and ~/.claude, or the whole container with `vzdump`.

Phone alerts with ntfy (optional)

Agentic-Errors.txt only helps when you're at the container. If you also want a push on your phone, `agentic-errorlog` can send one through ntfy every time it writes an entry. It stays off until you set a topic. The push is short (how many problems, the first few lines, and which machine); the file stays the full record.

1. **Pick a topic name that's hard to guess.** On the public ntfy.sh server a topic has no password: anyone who knows its name can read your alerts and post fake ones. Use something long and random, such as `claude-alerts-` followed by the output of `openssl rand -hex 8`. Avoid your name, the hostname, or anything obvious like `claude-errors`.
2. **Subscribe on your phone.** Install the ntfy app (Android or iOS), tap **+**, and enter the same topic (and your server, if you use your own).
3. **Give the topic to the container.** Answer the ntfy prompt when you run the deploy script on the Proxmox host, or set it later with `sudo nano /etc/agentic/ntfy.conf`.
4. **Test it.** Run `sudo agentic-errorlog --test-ntfy`. A test push should arrive within seconds.

The settings live in `/etc/agentic/ntfy.conf`, which only root can read because it can hold a token:

Setting	Meaning
`NTFY_TOPIC`	Topic name, or a full URL such as `https://ntfy.example.com/my-topic`. Empty turns alerts off.
`NTFY_SERVER`	Server for a bare topic name; default `https://ntfy.sh`. Change it if you run your own ntfy server.

Setting	Meaning
`NTFY_TOKEN`	Access token (`tk_...`) for a server or topic that requires a login; leave empty otherwise.

For more privacy, reserve the topic with an ntfy.sh account or run your own server, and use an access token. To stop alerts, empty `NTFY_TOPIC`.

Best practices with Claude

Treat Claude like a fast, capable colleague with shell access. Give it clear goals and a way to check its work, and review what it changes.

- **Use git for everything.** Commit before a big task, so a bad change is one `git reset` away. Read diffs before you push.
- **Plan first on big tasks.** Press Shift+Tab for plan mode, or use `/superpowers:brainstorm` and then `/superpowers:write-plan`.
- **Give it a way to verify.** Tests, a build command, or the webapp-testing skill for UIs let Claude check itself.
- **Keep secrets out of reach.** Put credentials in `.env` files or a `secrets/` folder; the deny rules stop Claude reading those, `*.pem` and SSH keys. Never paste a token into the chat.
- **Write down project rules.** Add a CLAUDE.md to each project (build, test and style commands); keep the shared /project/CLAUDE.md short.
- **Watch auto mode.** It approves most actions on its own, behind a safety classifier and the deny rules. For risky work (production data, real infrastructure), switch to default mode with Shift+Tab, or use `/permissions`.
- **Don't run Claude as root.** The `claude` command refuses to run as root here; keep it that way.
- **Manage context.** Use `/clear` between unrelated tasks, `/compact` on long ones, and `/effort` to spend more or less thinking.
- **Review plugin and MCP additions.** Anything you install runs as your user; add only sources you trust.

Security model

The container is the security boundary. Inside it, Claude runs as a normal user with no Docker group and sudo behind a password, but treat anything inside as reachable by the agent: what matters most is that container root is not Proxmox root.

Layer	What the script does	Why
Container	Unprivileged by default (asked at deploy), with only `nesting=1`: no Docker, no keyctl and the default AppArmor profile	Container root maps to an unprivileged host UID, so an escape doesn't own the host

Layer	What the script does	Why
Claude user	Normal account; root is blocked from running `claude`	Limits what an agent mistake touches
Claude permissions	Auto mode, plus deny rules for secrets, `dd`, `mkfs`, `rm -rf /`, `sudo` and `su`	Claude can't read credentials or escalate on its own
SSH	Root and the user log in with a password or key; at most 4 tries per connection	Convenient; keep port 22 on your LAN
sudo	Optional, password required; `agentic-doctor` warns if a NOPASSWD rule appears	Admin work stays with you; Claude can't type the password
Web UI	Runs as the user; own login; listens on the machine only unless you chose LAN at deploy, and starts only after setup is finished	Plain HTTP on port 3001: use an SSH tunnel, or keep LAN access to a trusted network or behind a TLS reverse proxy
Phone alerts	Off unless you set an ntfy topic; settings file is root-only	A guessable topic would let others read your alerts
Updates	Weekly, with health check	Keeps the OS, dev tools, Claude Code and the web UI patched

Why no Docker? Docker in an LXC needs extra container features (keyctl, an unconfined AppArmor profile) that weaken the container, and the `docker` group is root-equivalent inside it. Everything here runs natively instead, so the container keeps Proxmox's default confinement.

Further hardening you can add:

- Turn on the Proxmox firewall for the container, allowing only 22 from your LAN (and 3001 if you chose LAN access for the web UI).
- Use SSH keys, and set PasswordAuthentication no in /etc/ssh/sshd_config.d/00-agentic.conf once they work.
- If you answered N to unprivileged, rebuild as unprivileged; nothing in this setup needs a privileged container.

Troubleshooting

Start with `~/Agentic-Errors.txt` if it exists, then `sudo agentic-doctor`; each FAIL line names the part to look at. The comment block at the top of the deploy script has a "Where things live" list of every file it writes, and comments through the script explain each step.

Symptom	Check or fix
Deploy failed	Read the host log in `/var/log/agentic-deploy/`; the `-provision.log` copy shows the failing line. Then `pct stop <id>; pct destroy <id>` and re-run.
`claude: not run as root`	Expected. Use `su - <user>`, then `claude`.

Symptom	Check or fix
`claude: command not found` as the user	Run `sudo agentic-setup-user`.
Web UI won't open	It's local only unless you chose LAN: use the SSH tunnel above. If the tunnel doesn't work either, check `journalctl -u cloudcli -n 50`, then `sudo agentic-update` (it rebuilds native modules).
Doctor: "sudo asks for a password" WARN	A NOPASSWD rule (or `Defaults !authenticate`) lets the user use sudo without a password, which an auto-mode agent could use. `sudo -l -U <user>` shows it; remove it with `sudo visudo` or by editing the file in /etc/sudoers.d/.
Can't SSH in	Check the password, then from `pct enter <id>` run `systemctl status ssh`.
`sudo` asks for a password Claude can't give	Expected. Run the command yourself.
Harmless lines in the install log	`apparmor_parser ... Access denied`, `rehash: skipping ca-certificates.crt` and the `sudo` alternatives warning can be ignored.
No ntfy push arrives	Run `sudo agentic-errorlog --test-ntfy`; it names the failure. Check the topic matches the one in the app, and the token if your server needs one.